PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT – 4

Case Study 4: Attendance Recovery Computation.

1. Problem Statement

Attendance is an important requirement for students in colleges. Students who have low attendance need to know how many additional classes they must attend to reach the required attendance percentage. Calculating this manually can be time-consuming and confusing. This project develops a Python program that calculates the student's current attendance percentage and determines the minimum number of consecutive classes the student must attend to achieve the required attendance percentage.

2. Objectives

1. To calculate the student's current attendance percentage.
2. To check whether the student meets the required attendance percentage.
3. To determine the minimum number of additional consecutive classes the student must attend to reach the required percentage.
4. To validate the attendance details entered by the user.
5. To display the attendance percentage and the number of additional classes required.

3. Assumptions

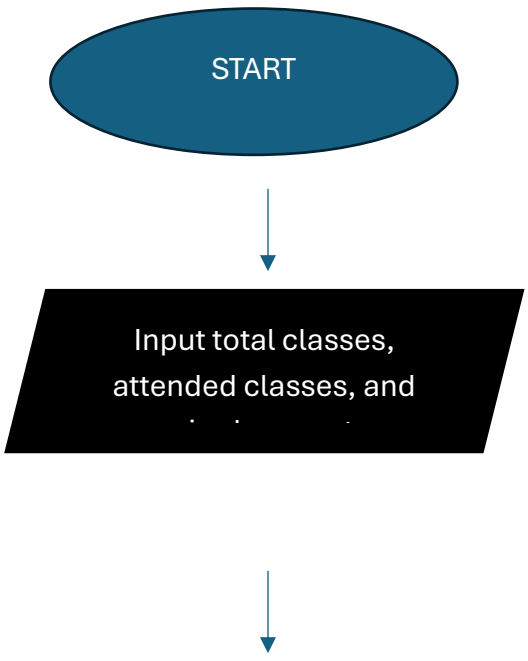
1. The total number of classes conducted must be greater than zero.
2. The number of classes attended cannot be negative or greater than the total classes conducted.
3. The required attendance percentage must be between 1% and 100%.
4. The student attends every additional class without missing any.
5. Each additional class increases both the number of classes attended and the total number of classes conducted by one.
6. The program calculates the minimum number of additional consecutive classes needed to reach the required attendance percentage.

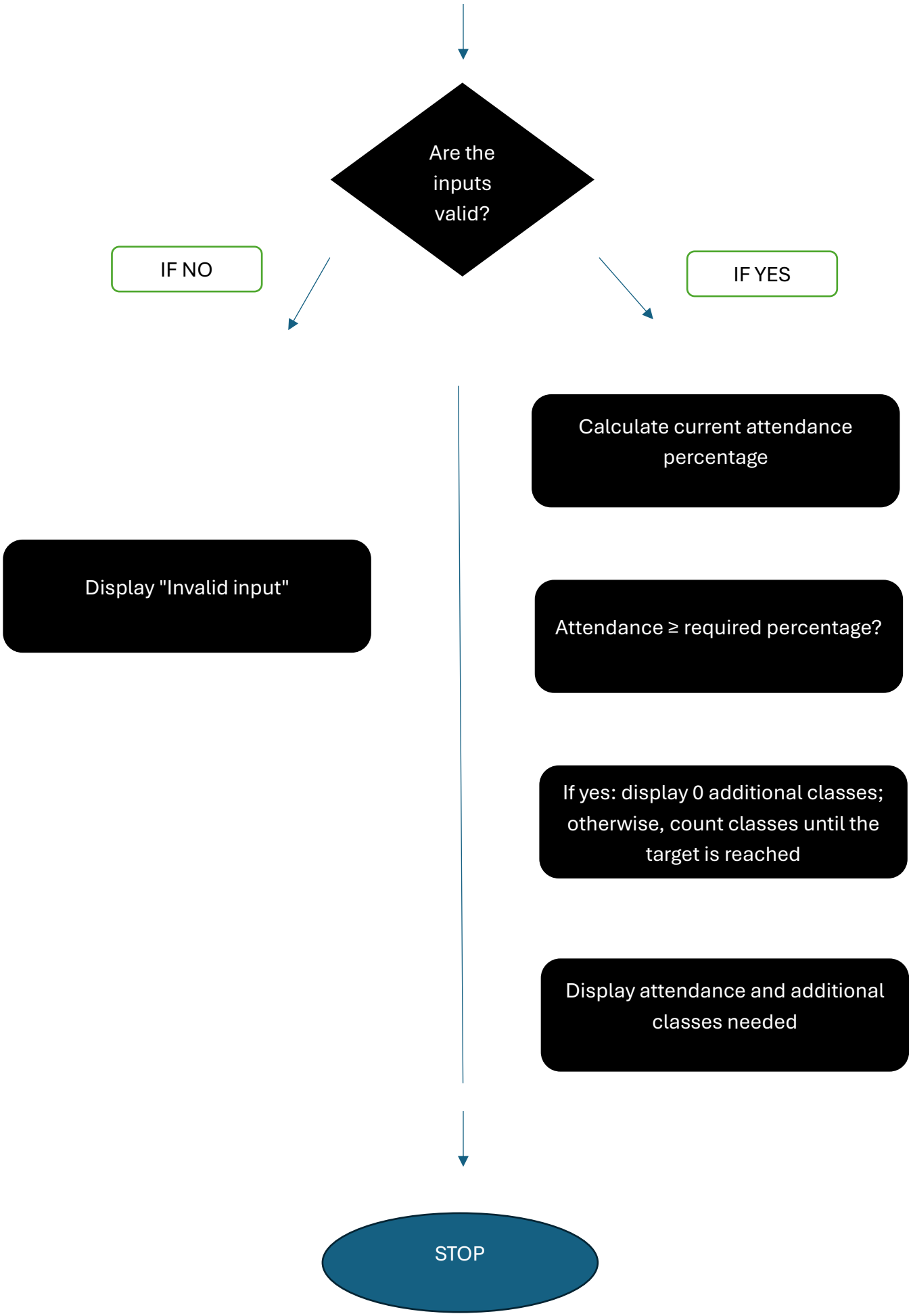
4. Algorithm

1. Start the program.
2. Input the total number of classes conducted.
3. Input the number of classes attended by the student.
4. Input the required attendance percentage.
5. Check whether the entered values are valid.
6. Calculate the current attendance percentage using the formula:

   Attendance Percentage = (Classes Attended / Total Classes Conducted) × 100
7. If the current attendance percentage is greater than or equal to the required percentage, display that the requirement is already achieved.
8. Otherwise, initialize the number of additional classes to zero.
9. Increase the number of additional classes until the attendance percentage reaches the required percentage, assuming the student attends every new class.
10. Display the current attendance percentage and the minimum number of additional classes required.
11. Stop the program.

5. Flowchart




Are the inputs valid?
IF NO
IF YES
Display "Invalid input"
Calculate current attendance percentage
Attendance ≥ required percentage?
If yes: display 0 additional classes; otherwise, count classes until the target is reached
Display attendance and additional classes needed
STOP

6. Pseudocode

START

INPUT total _classes

INPUT attended_ classes

INPUT required _percentage

IF total_ classes <= 0 OR attende _classes < 0 OR attended_ classes > total_ classes THEN

DISPLAY "Invalid attendance details"

ELSE IF required _ percentage <= 0 OR required _ percentage > 100 THEN

DISPLAY "Invalid required percentage"

ELSE

Current _percentage = (attended_ classes / total _classes) * 100

DISPLAY current _ percentage

IF current_ percentage >= required _ percentage THEN

  Additional _ classes = 0

  DISPLAY "Required attendance already achieved"

ELSE

  Additional _classes = 0

WHILE ((attended _ classes + additional_ classes) / (total_ classes + additional _ classes)) * 100 < required _ percentage

Additional _classes = additional_ classes + 1

END WHILE

DISPLAY additional _classes

END IF

END IF

STOP

7.**Source Code**

PYTHON-ACTIVITY-BASED-LEARNING > Attendance Recovery Computation.py > ...

```
total_classes = int(input("Enter total classes conducted: "))
attended_classes = int(input("Enter classes attended: "))
required_percentage = float(input("Enter required attendance percentage: "))

if total_classes <= 0 or attended_classes < 0 or attended_classes > total_classes:
    print("Invalid attendance details")

elif required_percentage <= 0 or required_percentage > 100:
    print("Invalid required percentage")

else:
    current_percentage = (attended_classes / total_classes) * 100

    print("Current attendance:", round(current_percentage, 2), "%")

    if current_percentage >= required_percentage:
        additional_classes = 0
        print("Required attendance already achieved")

    else:
        additional_classes = 0

        while ((attended_classes + additional_classes) /(total_classes + additional_classes)) * 100 < required_percentage:additional_classes += 1

        print("Additional classes required:", additional_classes)
```

8. Test Cases

Test Case	Input	Expected Output
TC1	Total classes = 100, Attended = 70, Required = 75%	Current attendance = 70%; Additional classes required = 20
TC2	Total classes = 80, Attended = 64, Required = 75%	Current attendance = 80%; Required attendance already achieved; Additional classes = 0
TC3	Total classes = 50, Attended = 30, Required = 80%	Current attendance = 60%; Additional classes required = 50
TC4	Total classes = 0, Attended = 0, Required = 75%	Invalid attendance details
TC5	Total classes = 100, Attended = 70, Required = 105%	Invalid required percentage

9. Output Screenshots



10. Conclusion

The Attendance Recovery Computation program calculates a student's current attendance percentage and determines the minimum number of additional consecutive classes required to reach the target attendance

percentage. It validates user inputs and uses conditional statements, loops, and arithmetic operations to perform the calculations. This project demonstrates how Python can be used to solve a practical problem faced by students.